di.unito.it
DIPARTIMENTO
DI INFORMATICA

TLN

intro to contextualized embeddings and Transformers

Daniele Radicioni

outline

- contextualized embeddings
- transformers networks: architecture
  - attention mechanism
  - sub-word tokenization
- training
  - pre-training
  - fine-tuning

di.unito.it
DIPARTIMENTO
DI INFORMATICA

credits

- Jurafsky, D. (2000). Speech & language processing. Pearson Education.
  - Chapter 6: 6.8
- Rothman, D., & Gulli, A. (2022). Transformers for Natural Language Processing. Packt Publishing Ltd.
  - Chapters 1, 2
- Mihai Surdeanu and Marco A. Valenzuela-Escárcega (2022) Deep Learning for Natural Language Processing
  - Chapter 12: 12.1, 12.2, 12.3

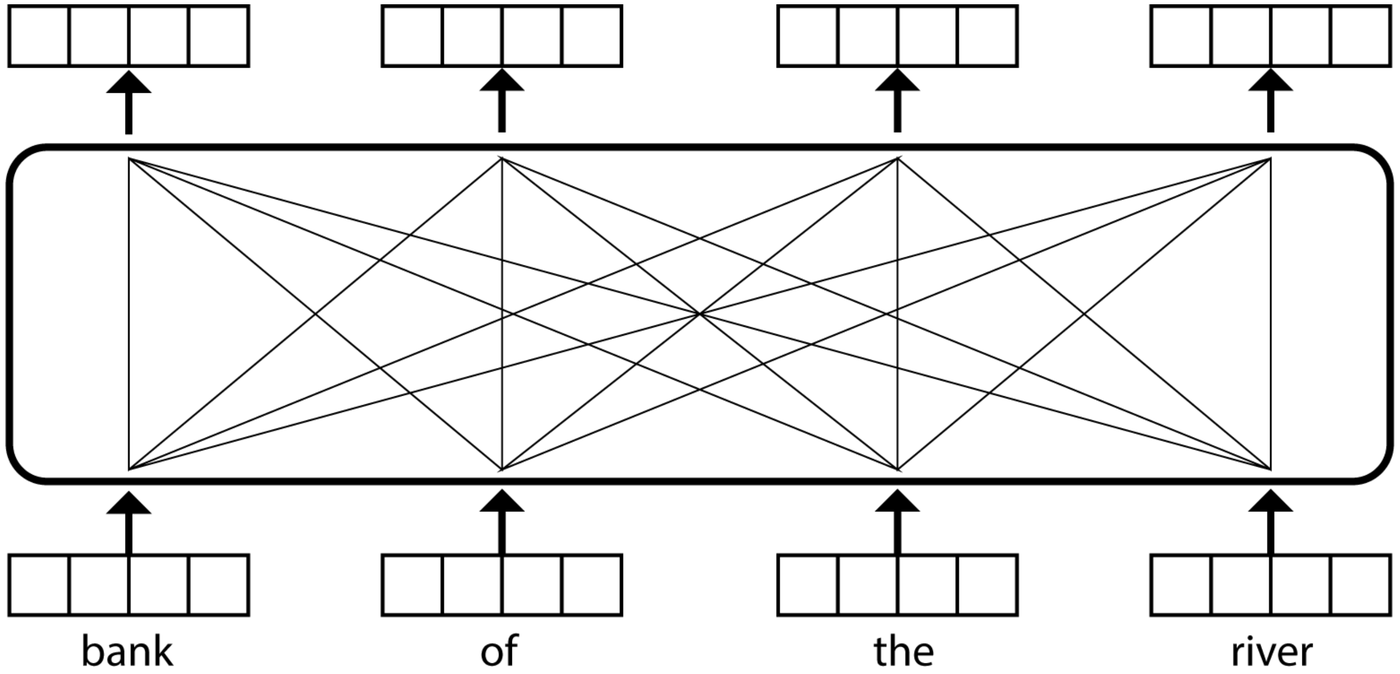
contextualized embeddings and transformers

meaning conflation problem

- distributional similarity algorithms conflate all senses of a word into a single embedding
  - for example, the word *bank* receives a single representation, while it can deliver different senses: financial (e.g., as in the *bank gives out loans*) or environmental (e.g., *bank of the river*)
- transformer networks (TNs) were designed to learn *contextualized embeddings*, that change according to the context in which terms appear
  - this means that *bank* receives different numerical representations in the above examples

contextualized embeddings

"... bank of the river ..."



- each word receives a contextualized embedding that is a weighted average of some context-independent input embeddings (similar, in essence, to word2vec)
  - since in the sentence we have a river, the resulting embedding of *bank* will lean towards natural environment
  - the same applies to all terms in the sentence
  - this produces distinct output embeddings for the different words in the text being processed

contextualized embeddings

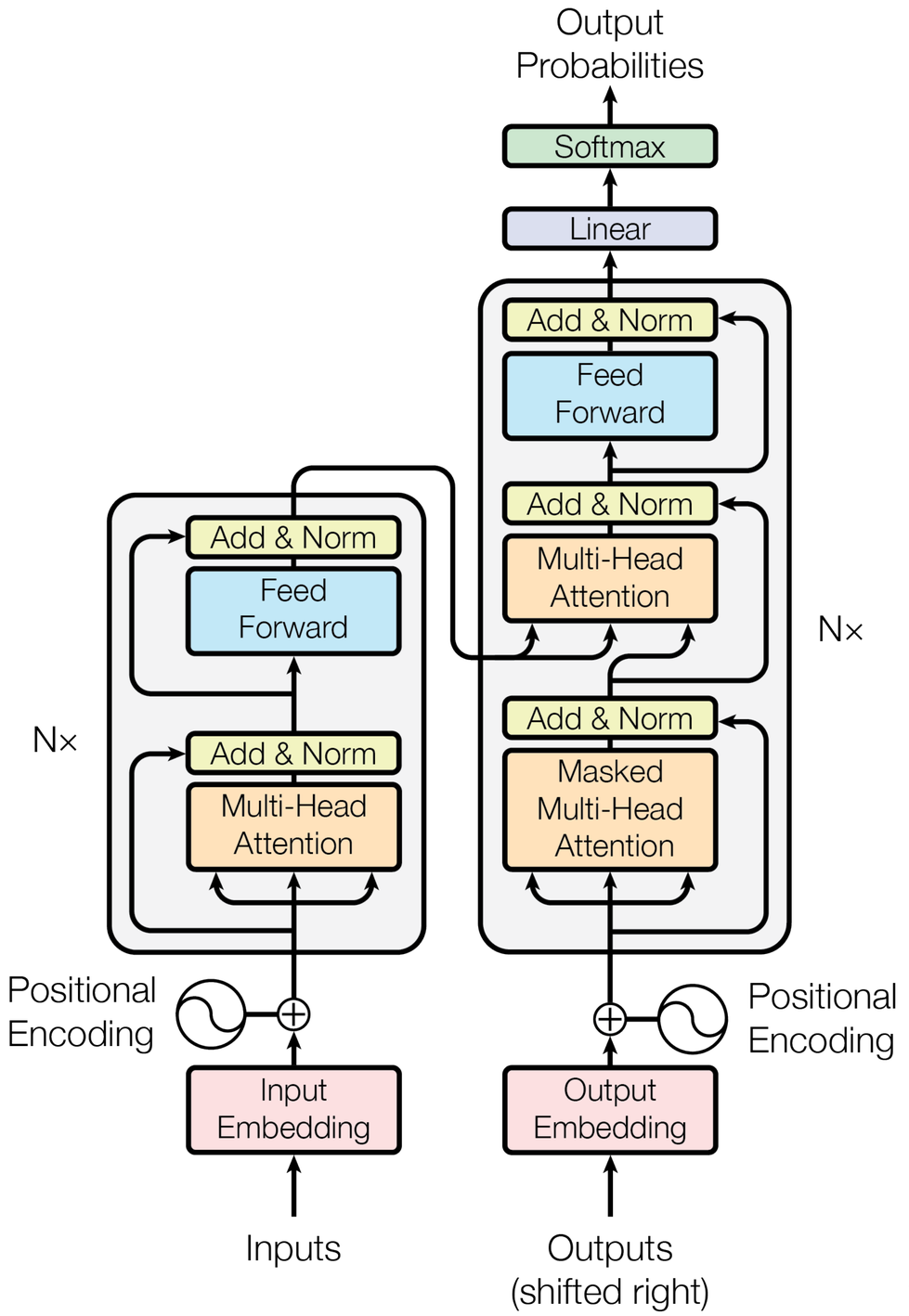
- transformer networks actually consist of a stack of multiple layers, where each layer implements a weighted average as previously illustrated
  - the output embeddings for layer i become the input embeddings for layer $i + 1$
  - layers typically range between 2 and 24
  - multiple layers allow these networks to learn more complex functions for assembling output embeddings (which has been empirically shown to yield more meaningful representations)

architecture

architecture intro

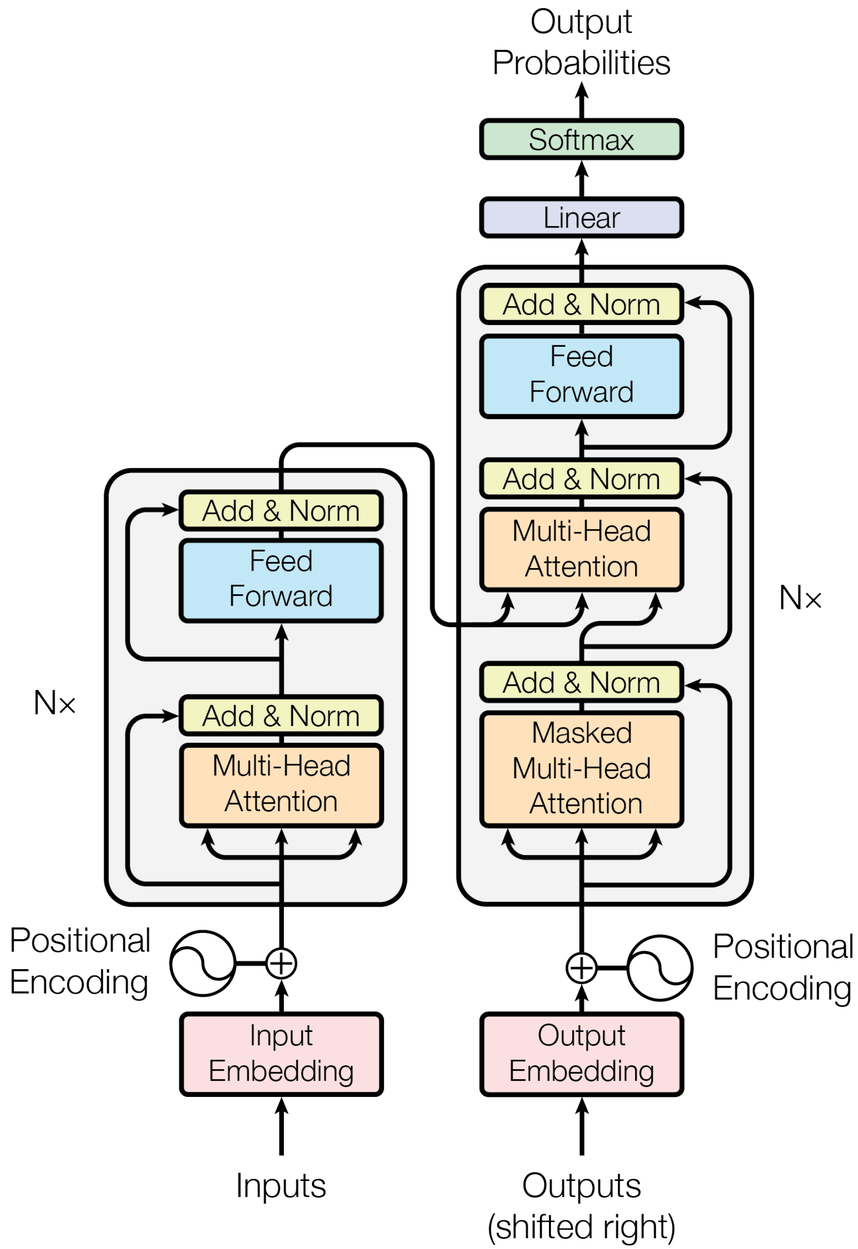
architecture

- the inputs enter the encoder side of the Transformer through an attention sub-layer and FeedForward Network (FFN) sub-layer. on the right, the target outputs go into the decoder side of the Transformer through two attention sub-layers and an FFN sub-layer.
  - there is no RNN, LSTM, or CNN. Recurrence has been abandoned.
- Attention has replaced recurrence, which requires an increasing number of operations as the distance between two words increases.
  - the attention mechanism will find how each word is related to all other words in a sequence
  - for each attention sub-layer, the original Transformer model runs not one but eight attention mechanisms in parallel to speed up the calculations.



architecture

- The original Transformer model is a stack of 6 layers
- The output of layer l is the input of layer $l + 1$until the final prediction is reached
- There is a 6-layer encoder stack on the left and a 6-layer decoder stack on the right



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Encoder and Decoder

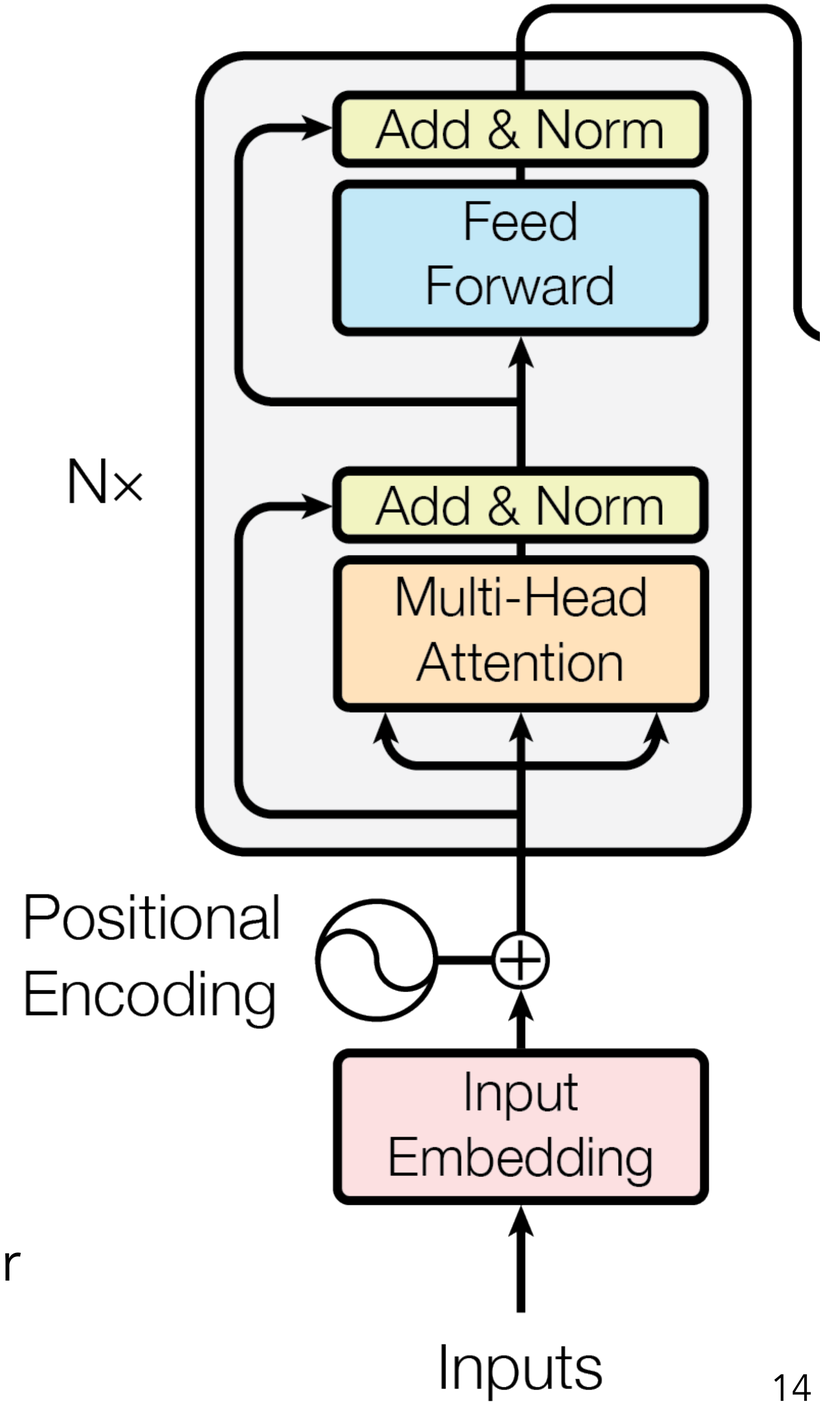
- the encoder is composed of a stack of N = 6 identical layers. each layer has two sub-layers:
  - the first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network.
- a residual connection is set around each of the two sub-layers, followed by layer normalization.
  - that is, the output of each sub-layer is LayerNorm(x+ Sublayer(x)), where Sublayer(x) is the function implemented by the sub-layer itself.
  - to facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{model} = 512$.

Encoder and Decoder

- the decoder is also composed of a stack of N = 6 identical layers.
- in addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack.
  - similar to the encoder, it employs residual connections around each of the sub-layers, followed by layer normalization.
  - the self-attention sub-layer in the decoder stack is modified so to prevent positions from attending to subsequent positions. this masking, combined with the fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

Encoder stack

- each layer of the encoder stack has this structure
  - each layer contains two main sub-layers: a multi-headed attention mechanism and a fully connected position-wise feedforward network
- a very efficient constraint: the output of every sub-layer of the model has a constant dimension, including the embedding layer and the residual connections (in the original Transformer architecture, the dimension $d_{model} = 512$).
  - all the key operations are dot products. the dimensions remain stable, which reduces the number of operations to calculate, reduces machine consumption, and makes it easier to trace the information as it flows through the model.



Encoder stack

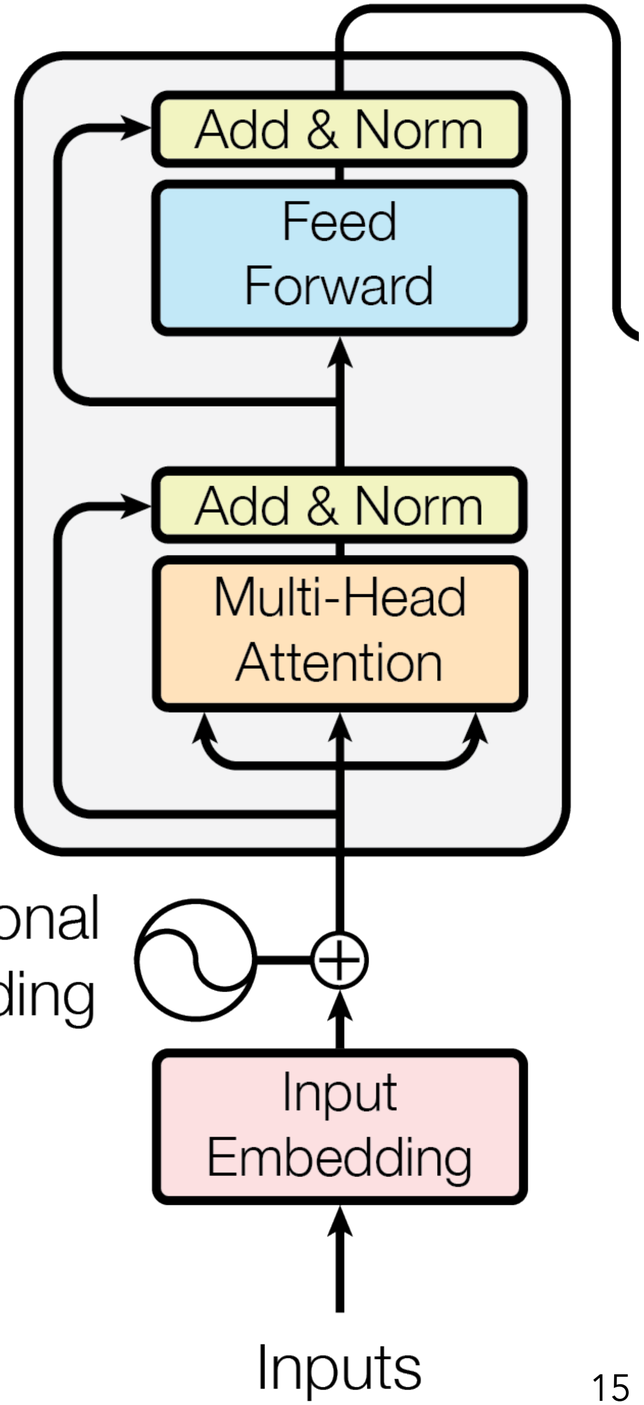
- eac
  - ea
    att
    fee
- a v
  of t
  em
  ori
  d_{mo}
  - all
    rer
    ca
    to trace the information as it flows through the model.

residual connection

instead of passing only the sublayer output onward, the model also keeps the original input and adds it back.

If a sublayer computes some transformation $F(x)$ of an input x, the residual output is $x + F(x)$.

This means the sublayer learns a correction or refinement to the input, rather than having to replace it completely. In practice, this helps preserve information, improves gradient flow during training, and makes deep stacks easier to optimize.



Encoder stack

- eac
  - ea
    att
    fee
- a v
  of t
  em
  orig
  d_{mo}
  - all
    rer
    ca
    to trace the information as it flows through the model.

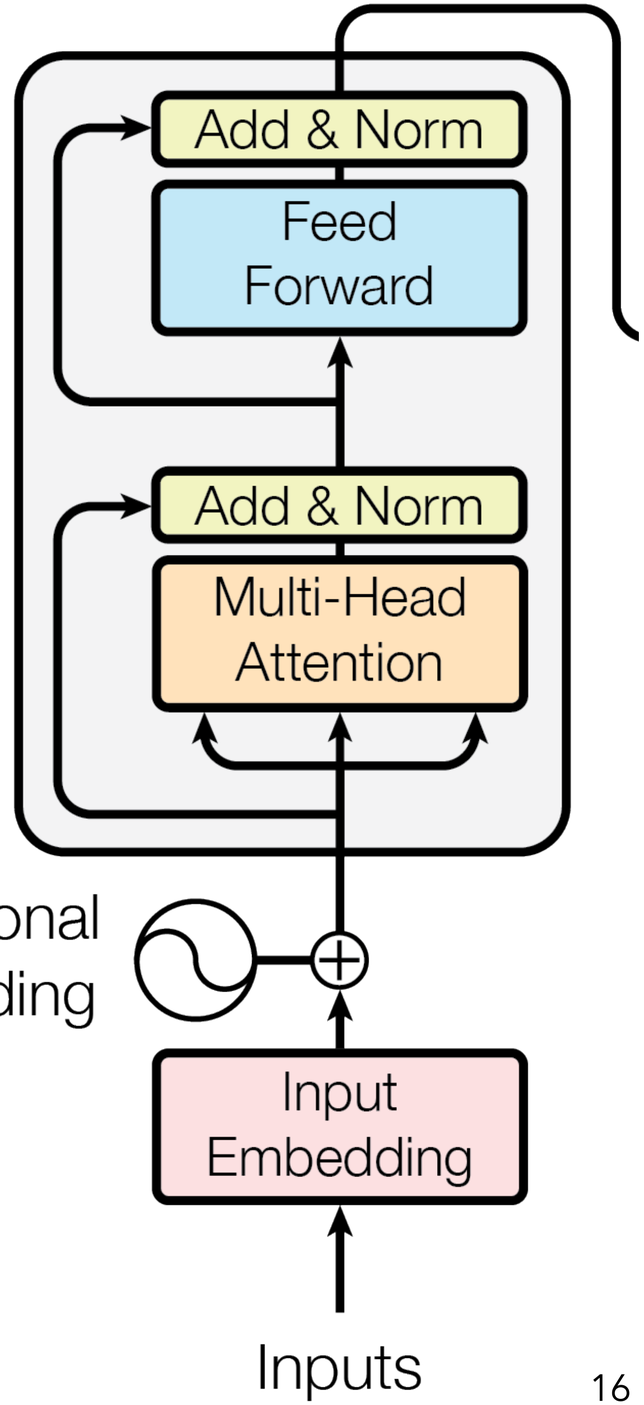
Add & Norm

After forming the sum $x + F(x)$, the model applies layer normalization: $\mathrm{LayerNorm}(x + F(x))$.

Layer normalization rescales and recenters the vector at each position so its components are kept in a stable numerical range. This makes training more stable and helps prevent activations from growing too large or becoming poorly scaled across layers.

The full pattern in the encoder block is:

$$x \longrightarrow F(x) \longrightarrow x + F(x) \longrightarrow \mathrm{LayerNorm}(x + F(x)) .$$



architecture

- each layer implements a sequence of five operations
  - the first operation adds positional information to the input embedding of each word in the input text: the embedding of each word changes depending on its position in the input text
  - in the second operation, embeddings are generated that are a weighted average of the embeddings produced at the previous stage (this step is called self attention, as the most meaningful parts of the data are identified and given higher weights in the weighted average)

architecture

- the subsequent operations are not that important conceptually, but they have been shown to have a significant contribution empirically.
  - for example, the 'Add and normalize' components sum up input and output embeddings from the previous component in the pipeline (e.g., the input embeddings to the self attention layer and the corresponding output embeddings computed through the weighted average), and normalize the results to avoid values that might be too large, which may negatively impact gradient descent

types of Transformers blocks (1)

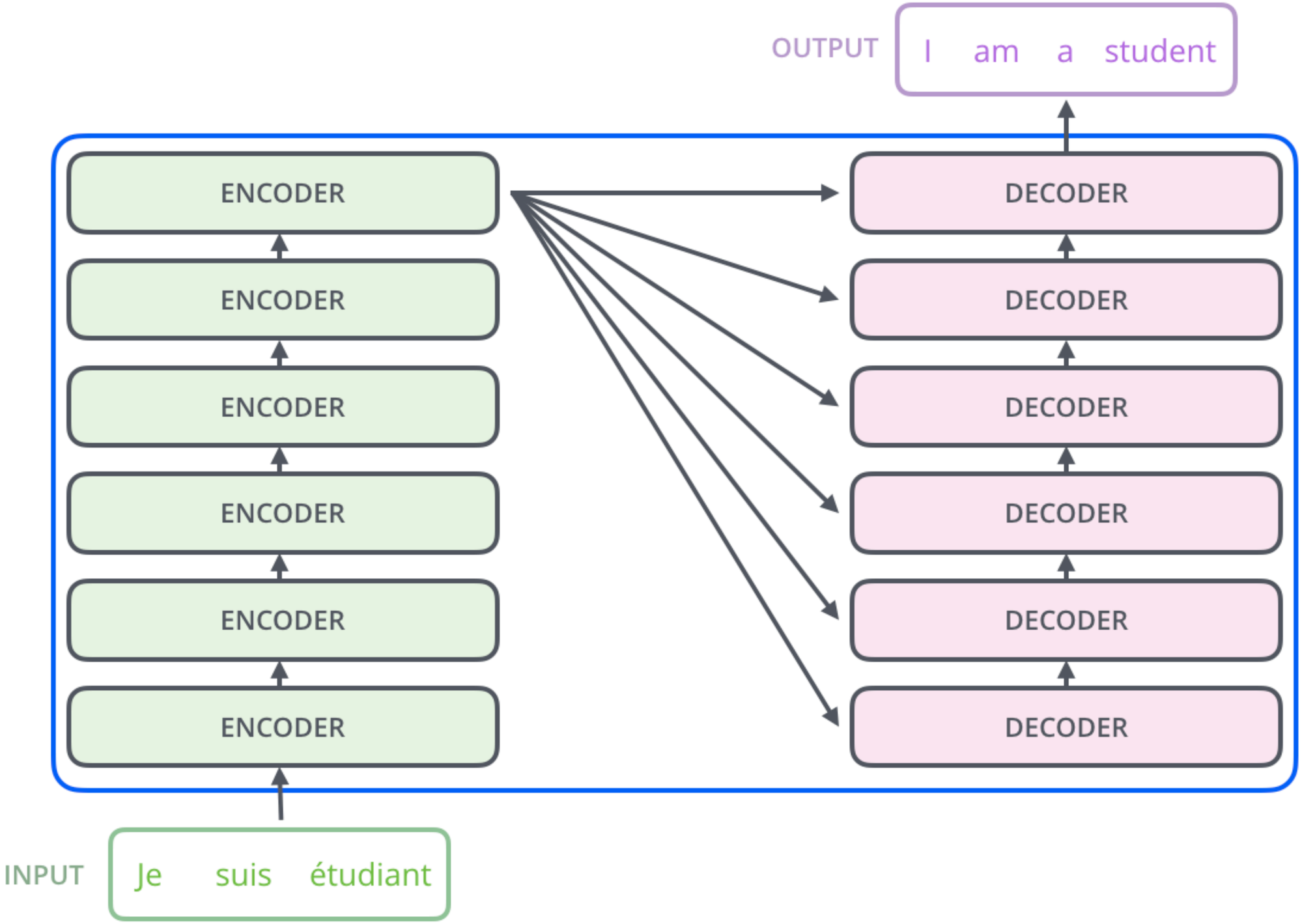
- ***Encoder-only*** models convert an input sequence of text into a numerical representation, where the representation computed for a given token depends both on the left and the right context. this is called *bidirectional attention*.
  - such a representation is suited for tasks like text classification or named entity recognition.
  - BERT and its variants belong to this class of architectures.

types of Transformers blocks (2)

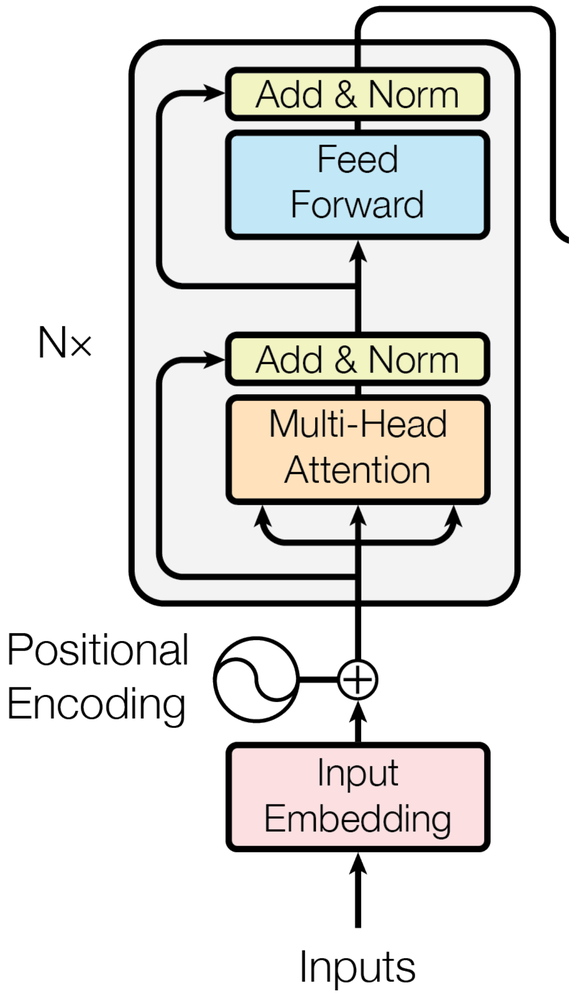
- ***Decoder-only*** models complete a given input sequence by iteratively predicting the most probable next word. this is often called causal or *autoregressive attention*.
  - GPT models belong to this class. the representation computed for a given token in this architecture depends only on the left context.
- ***Encoder-decoder*** models are used for modeling complex mappings from one sequence of text to another; these are suited for machine translation and summarization tasks.
  - the BART and T5 models belong to this class.

visual transformers

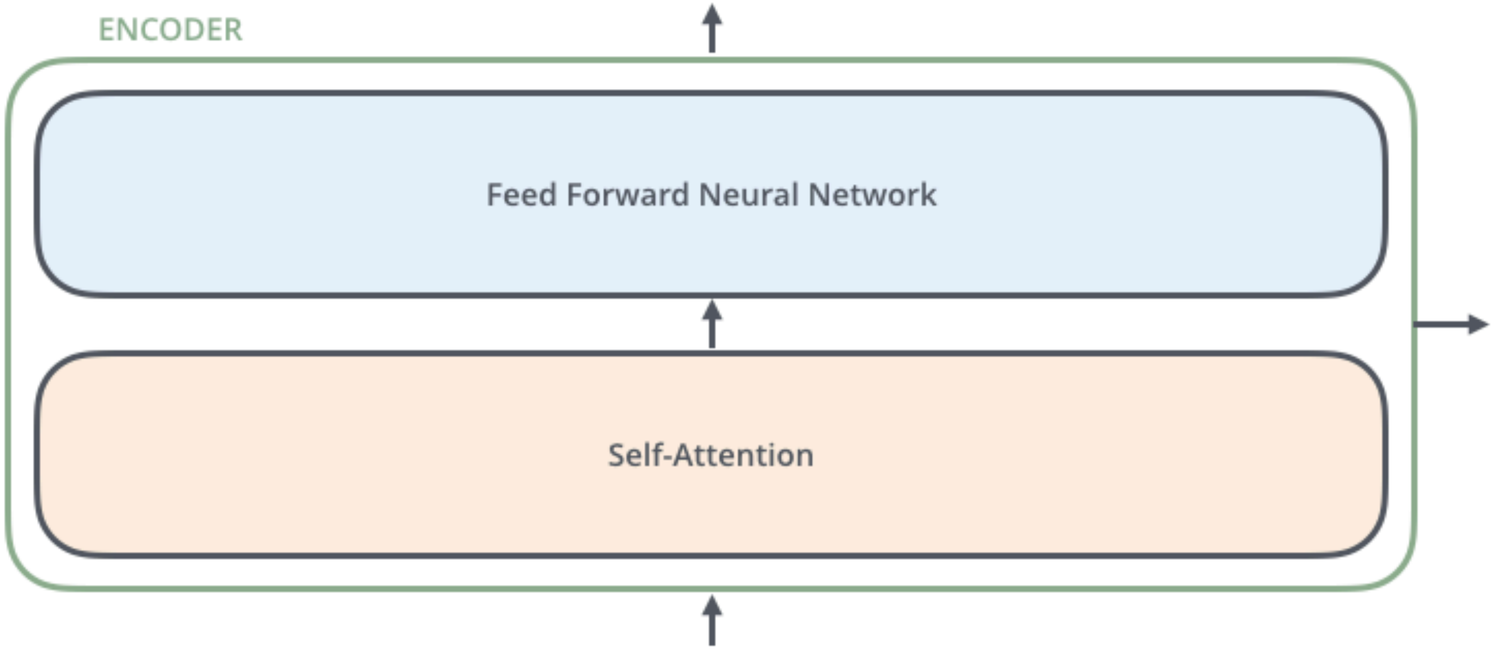
[the whole example is borrowed from https://jalammar.github.io/illustrated-transformer/]


OUTPUT
I am a student
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
DECODER
DECODER
DECODER
DECODER
DECODER
DECODER
INPUT
Je suis étudiant

encoder

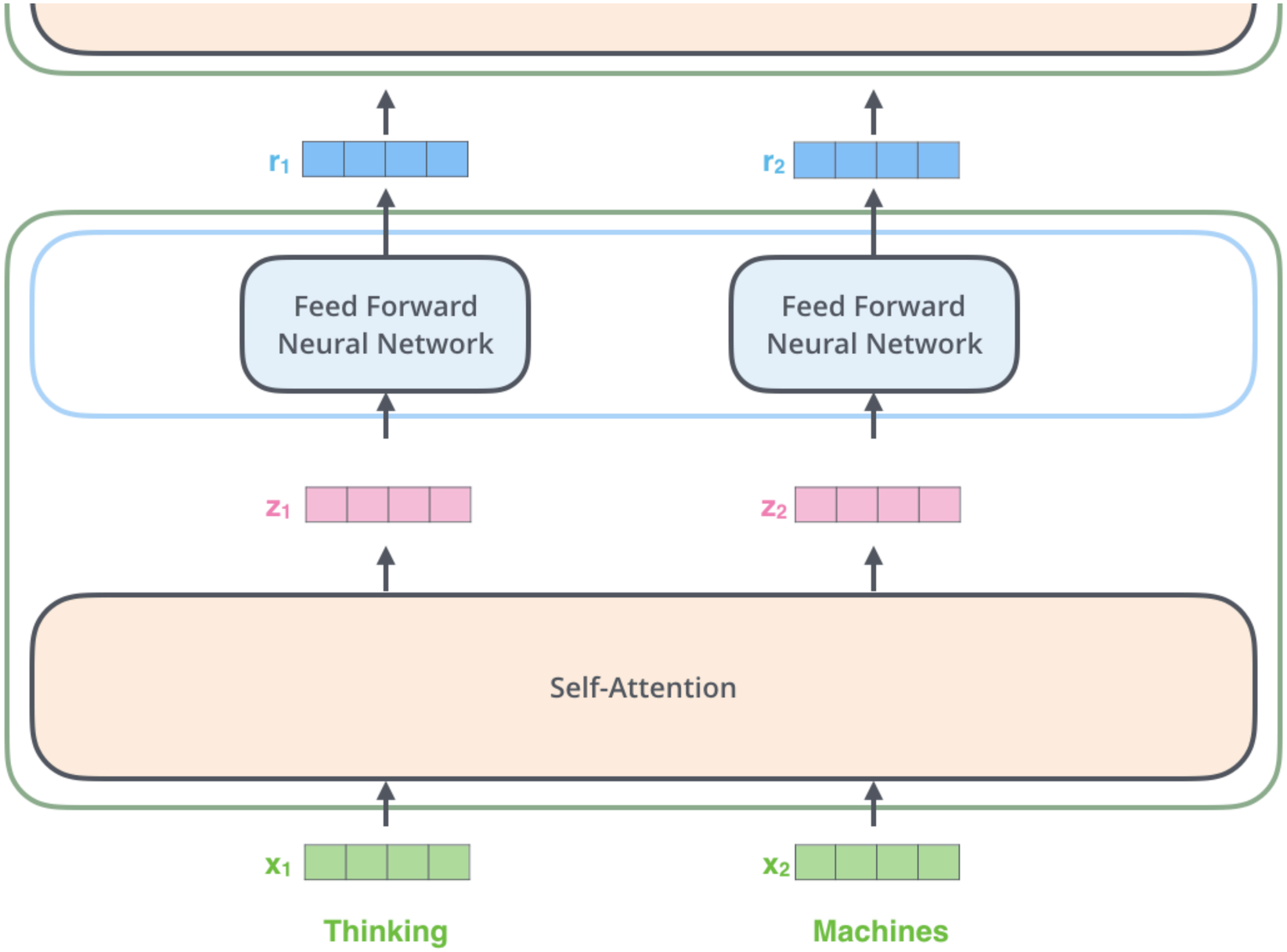


simplified encoder



- we turn each input word into a vector using an embedding algorithm
  - encoders receive a list of vectors each of size 512
  - the size of this list is a hyper-parameter we can set; it ideally represents the length of the longest sentence in the training dataset.

- the encoder takes a list of vectors as input. it processes this list by passing these vectors
  - into a ‘self-attention’ layer,
  - then into a feed-forward neural network, then sends out the output upwards to the next encoder.



the Feed-Forward Sublayer

- it adds non-linearity and feature transformation after the model captures relationships between tokens through attention. it has two main goals:
  - helps the model learn abstract features, e.g., syntactic roles, semantic classes, or even latent categories like verb types or named entities.
  - introduces non-linearity, enabling the model to capture more complex patterns than attention alone can.
- it includes:
  - a first linear layer: expands the dimensionality (e.g., from 768 to 3072)
  - a non-linear activation (typically ReLU or GELU)
  - a second linear layer: projects back to the original dimension (e.g., from 3072 to 768)

the Feed-Forward Sublayer

- the FF sublayer, like the rest of the Transformer, learns through backpropagation — but not from token-level labels (in general).
  - it gets indirect supervision from the model's overall training objective.
- for example, a training objective may be the Masked Language Modeling (as in BERT):
  - Input: "The [MASK] sat on the mat."
  - Target: Predict "cat".
  - Loss is computed only at the masked token, but the gradient flows through the entire network, including the FFNs for all tokens.

the Feed-Forward Sublayer

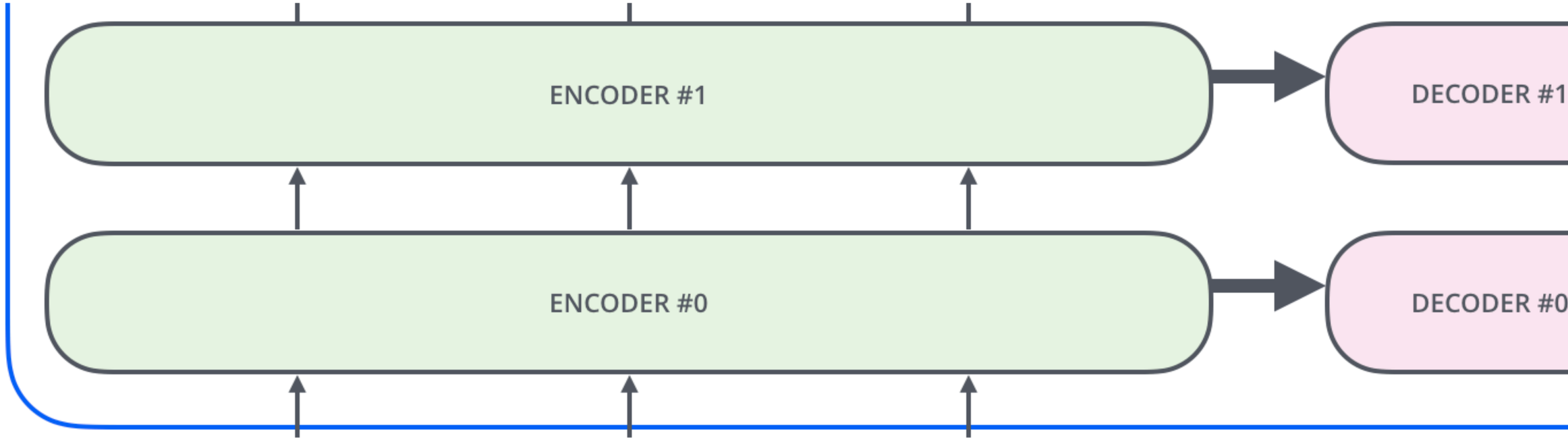
- this layer is charged to learn intermediate transformations of token representations that are useful for the final objective. these can include:
  - encoding syntactic roles: subject, object, modifier
  - highlighting semantic roles: agent, patient, location
  - emphasizing salient features: animacy, polarity, tense
  - creating latent clusters of similar functions

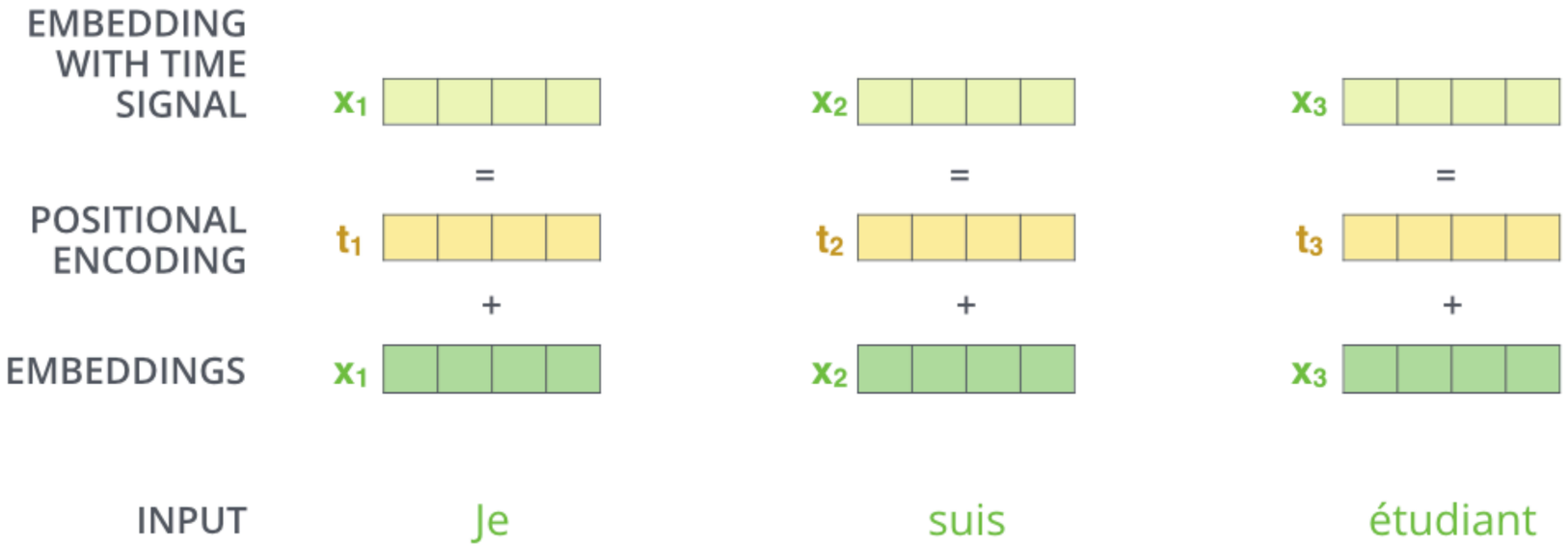
Self-Attention

- given the sentence "The animal didn't cross the street because it was too tired."
  - what does "it" in this sentence refer to? Is it referring to the street or to the animal?
- self-attention is the device that allows us to associate "it" with "animal"
  - we can think of self-attention as telling 'who talks to whom'

positional encoding

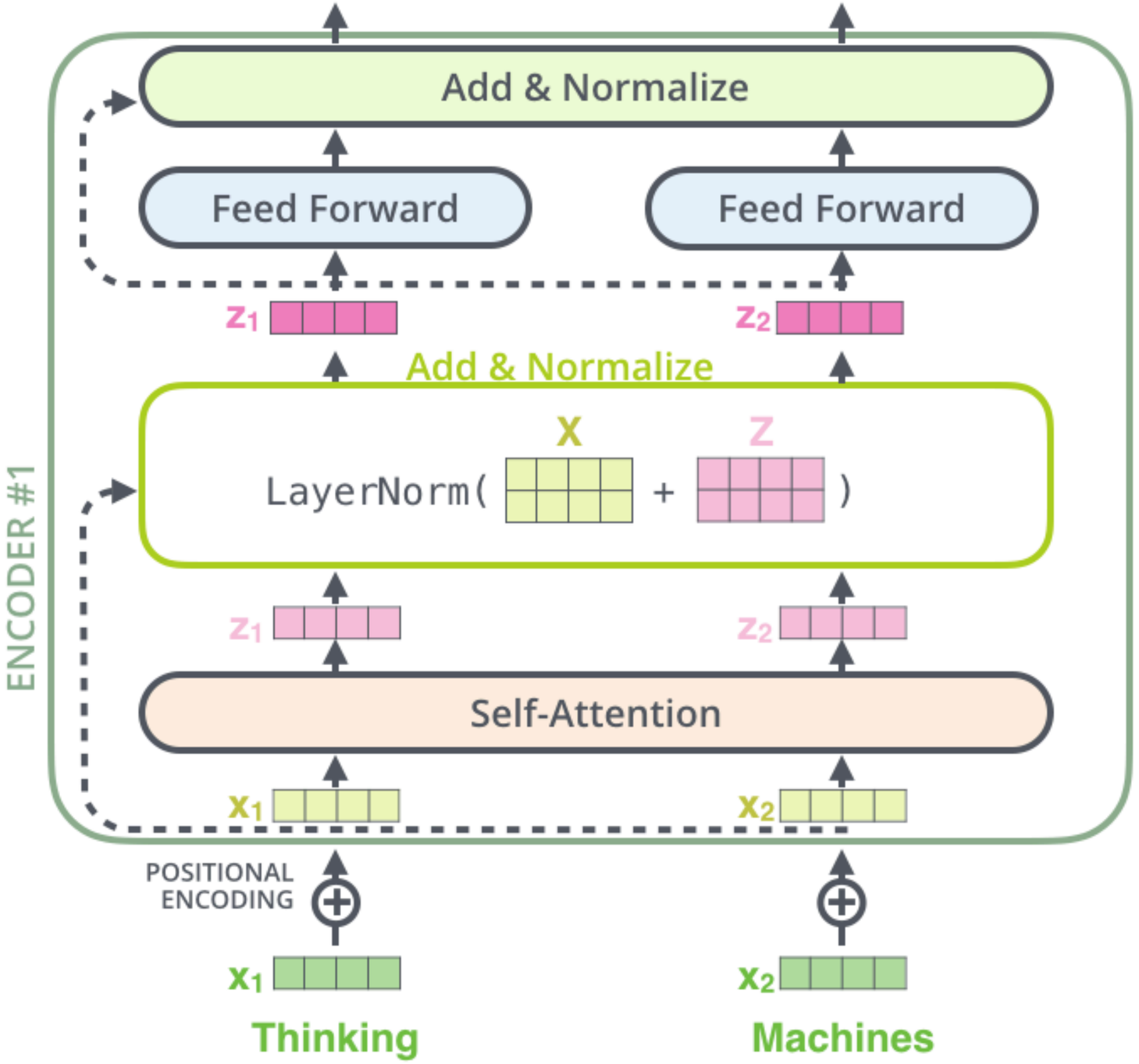
- it is a way to account for the order of the words in the input sequence
  - the transformer adds a vector to each input embedding. these vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence.


ENCODER #1
DECODER #1
ENCODER #0
DECODER #0


EMBEDDING WITH TIME SIGNAL
POSITIONAL ENCODING
EMBEDDINGS
x1
x2
x3
t1
t2
t3
=
+
INPUT
Je
suis
étudiant

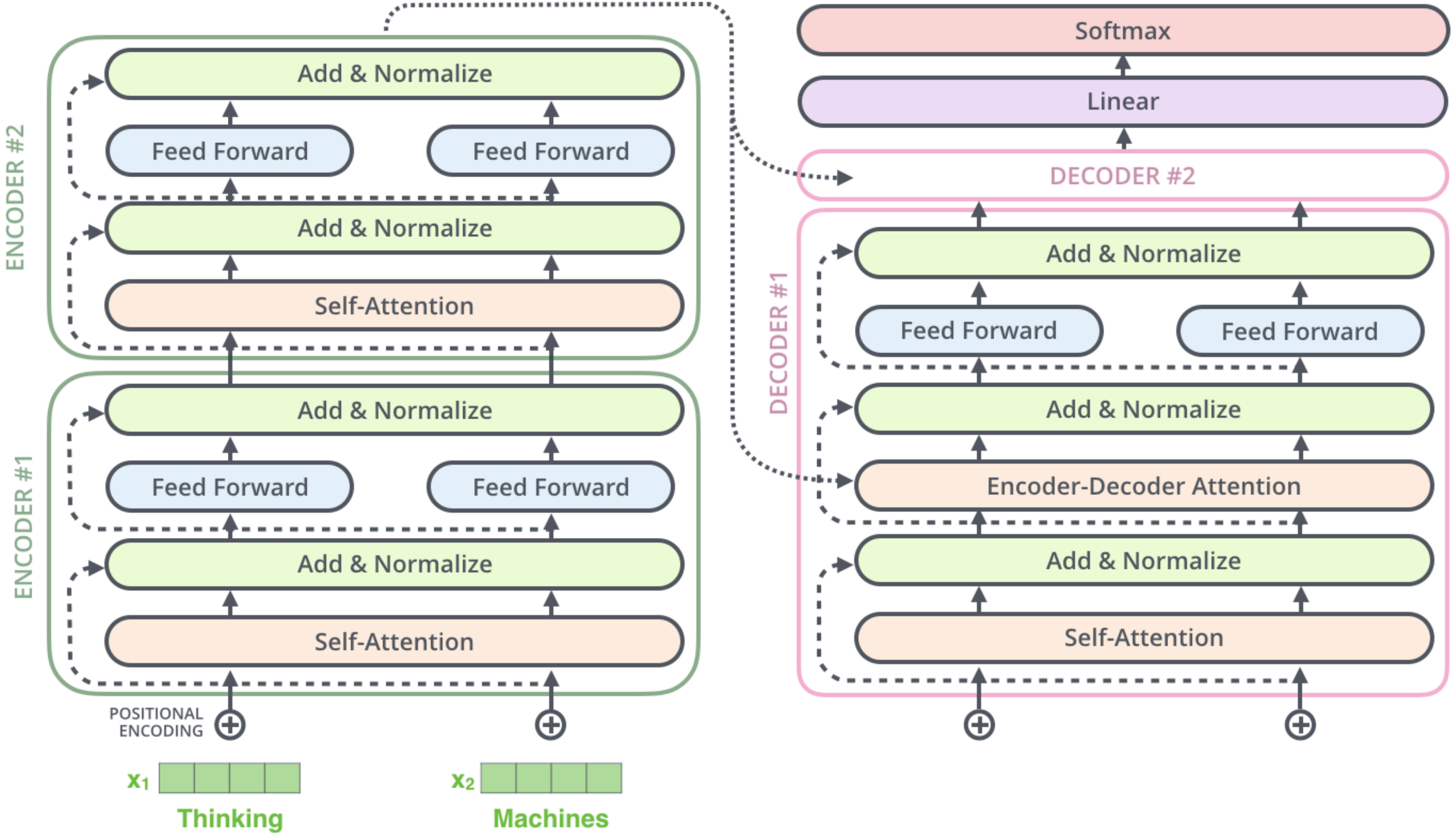
residuals

- each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.



decoder

- the output of the top encoder is then transformed into a set of attention vectors K and V.
  - these are to be used by each decoder in its "encoder-decoder attention" layer, which helps the decoder focus on appropriate places in the input sequence
- the Encoder-Decoder Attention layer works just like multi-headed self-attention, except it creates its Queries matrix from the layer below it, and takes the Keys and Values matrix from the output of the encoder stack.


Softmax
Linear
DECODER #2
DECODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Encoder-Decoder Attention
Add & Normalize
Self-Attention
ENCODER #2
ENCODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
POSITIONAL ENCODING
x1
x2
Thinking
Machines

Linear and Softmax Layers

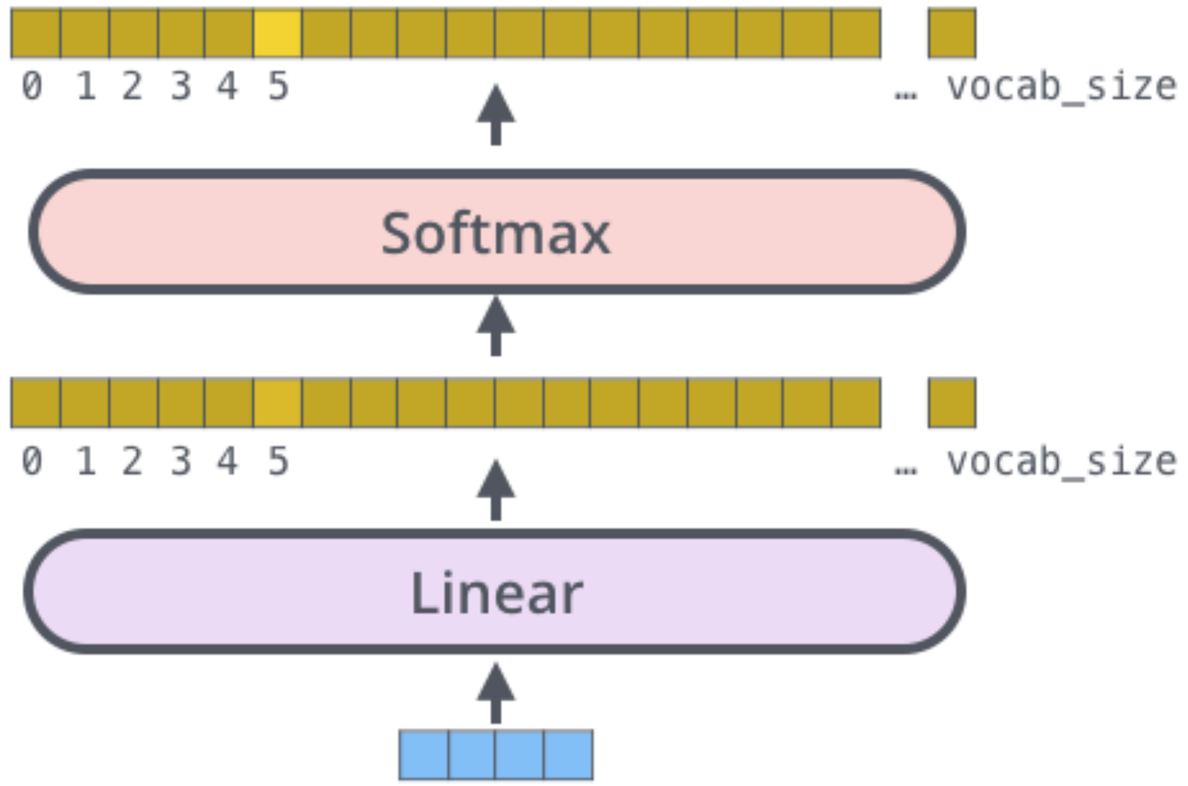
- the decoder stack produces as output a vector of floats, that is turned into a word by the final Linear layer, which is followed by a Softmax Layer.
- the Linear layer is a fully connected neural network that projects the vector produced by the stack of decoders, into a much larger vector called a logits vector.
  - if the model has learned from the training set 10,000 unique English words ("output vocabulary") the logits vector would be 10,000 cells wide (with each cell corresponding to the score of a unique word)
- the softmax layer then turns those scores into probabilities (all positive, all add up to 1.0).
  - the cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step.

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (**argmax**)

5



Training

- so far, an entire forward-pass process through a trained Transformer has been shown
- at training time, an untrained model would go through the exact same forward pass.
  - but since we are training it on a labeled training dataset, we can compare its output with the actual correct output.

- let's assume our output vocabulary only contains six words ("a", "am", "i", "thanks", "student", and "<eos>" (short for 'end of sentence'))
- we can use a vector of the same width to indicate each word in our vocabulary.
  - this also known as one-hot encoding

Output Vocabulary

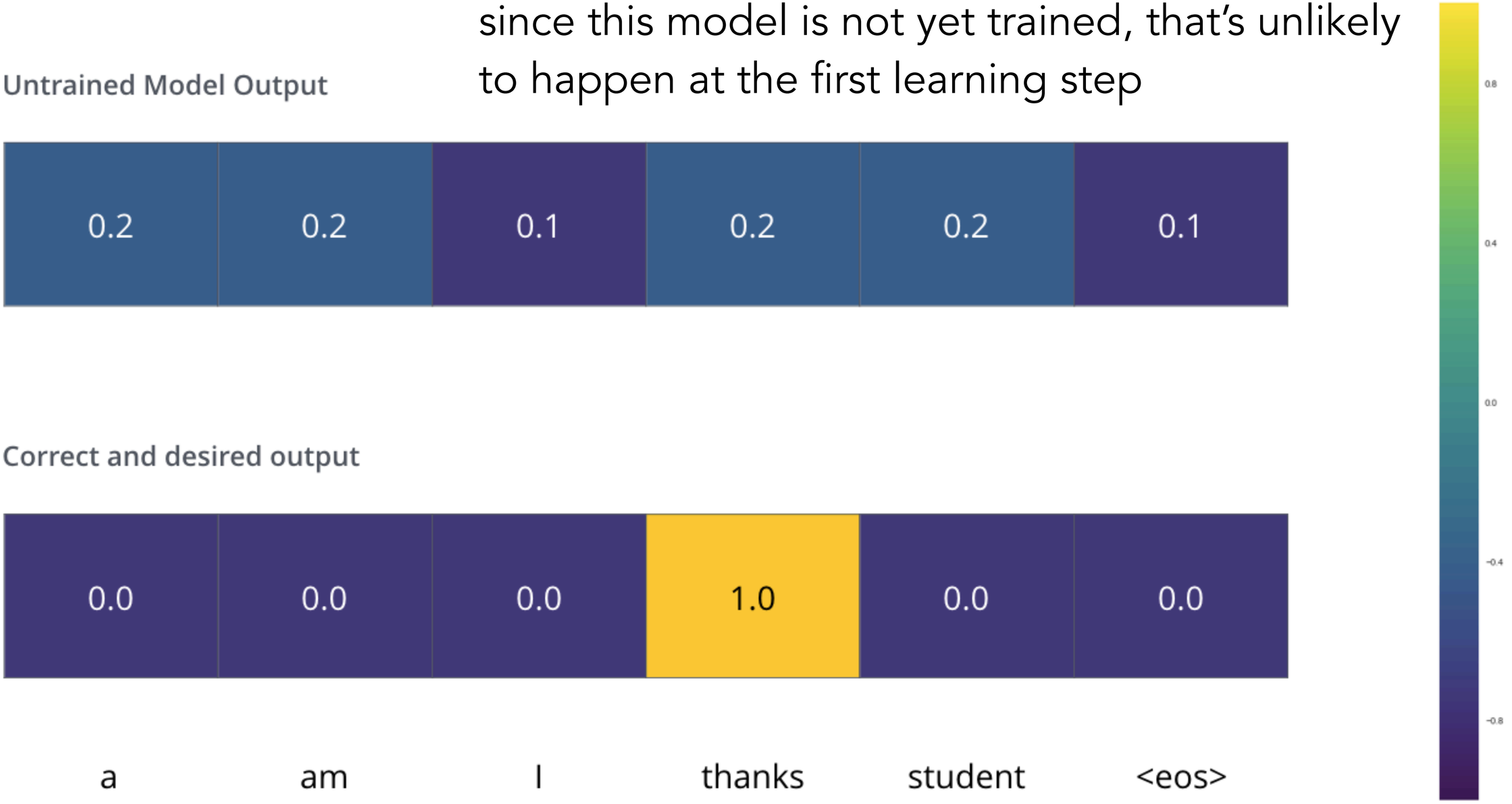
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"

0.0	1.0	0.0	0.0	0.0	0.0

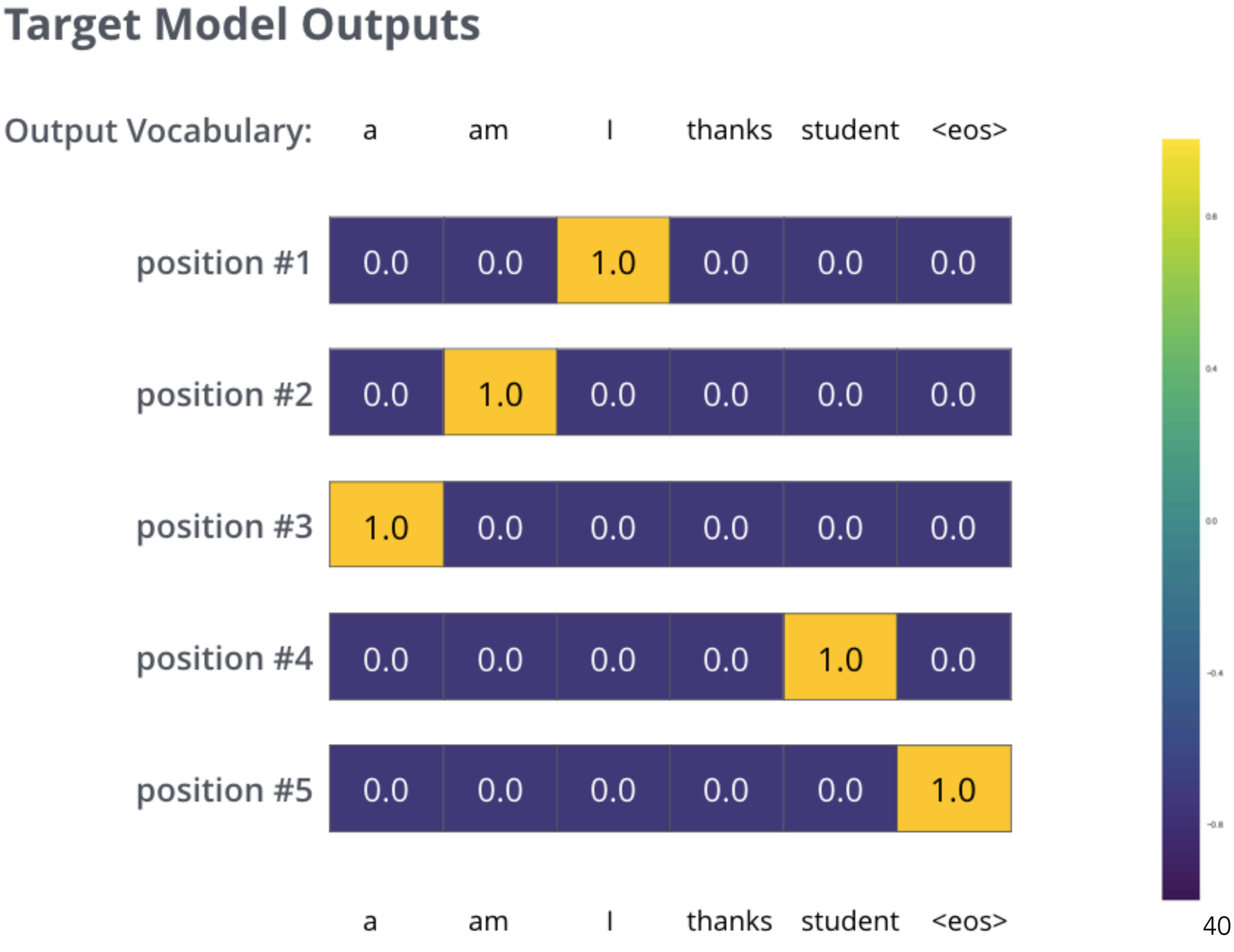
Loss function

we want the output to be a probability distribution indicating the word "thanks". but since this model is not yet trained, that's unlikely to happen at the first learning step

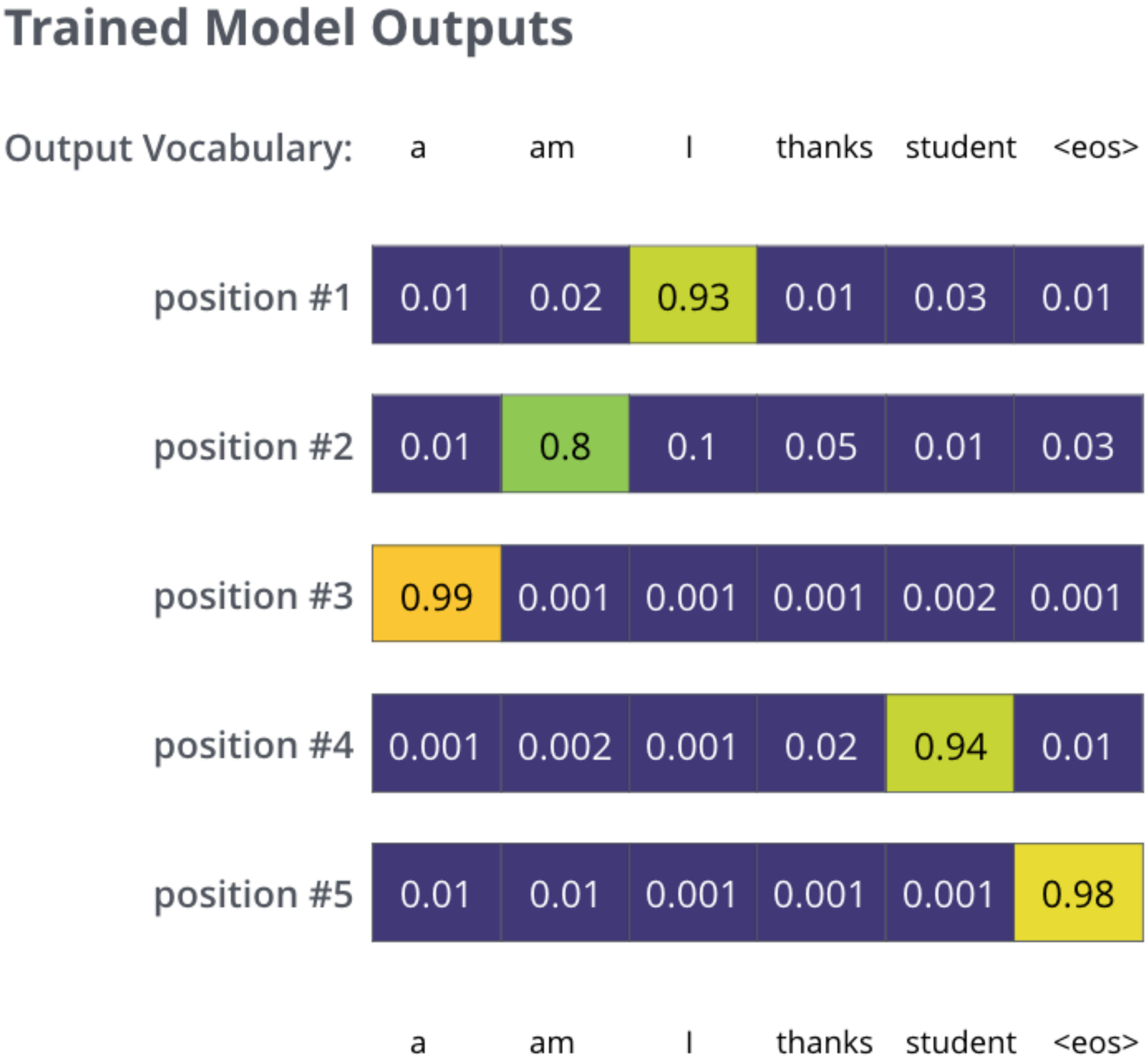


Loss function

- how to compare two probability distributions?
  - we simply subtract one from the other.



Loss function



di.unito.it
Dipartimento di Informatica

positional embeddings

positional embeddings

- let us consider the sentence "*Bank of America financed a repair of the river bank*"
  - the network should realize that the first instance of *bank* refers to a financial institution because it is closer to the word *financed*, and distant from the word *river*.
  - modeling such proximity information requires keeping track of positional information, i.e., where words occur in the input texts.

positional embeddings

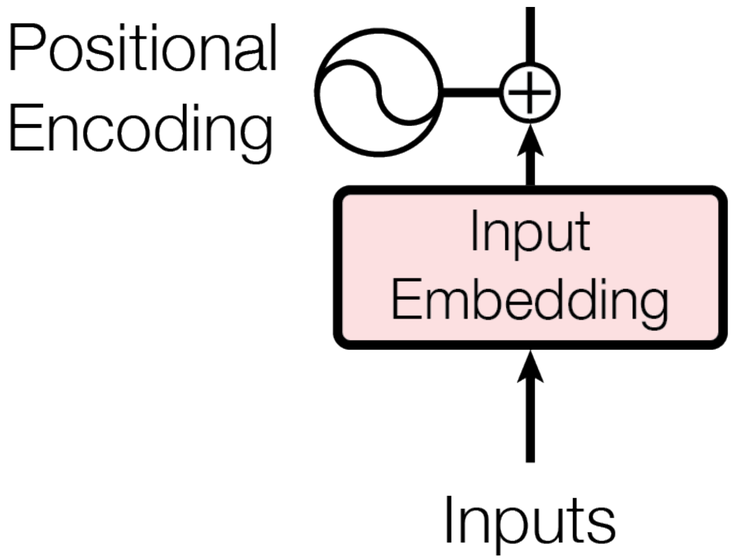
- before transformers, neural approaches typically modeled positional information through a separate numerical representation
  - the input embedding of a word w_i is a concatenation of two vectors, $\mathbf{x}_i = [\mathbf{w}_i; \mathbf{p}_i]$, where $\mathbf{w}_i$ is a regular (e.g., word2vec) word embedding; and $\mathbf{p}_i$ is a vector of numerical representations of word positions
  - during the training, the network also learns numerical representations for word positions through the $\mathbf{p}$ vectors

positional embeddings

- for a word in position i in the input text, transformers networks generate a vector $\mathbf{p}_i$ which is unique for each position i
  - the input embedding $\mathbf{x}_i$ for word w_i is $\mathbf{x}_i = \mathbf{w}_i + \mathbf{p}_i$
  - this function is hard-coded

positional encoding

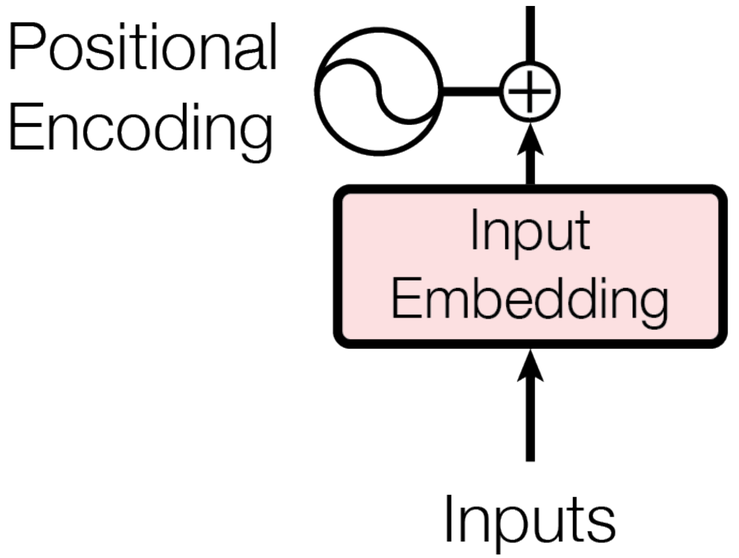
- the idea is to add a positional encoding value to the input embedding instead of having additional vectors to describe the position of a token in a sequence.
  - a method is needed to add a value to the $d_{model} = 512$ dimensions: for each word embedding vector, we need to find a way to provide information to i in the range $(0,512)$ dimensions of the word embedding vector



positional encoding

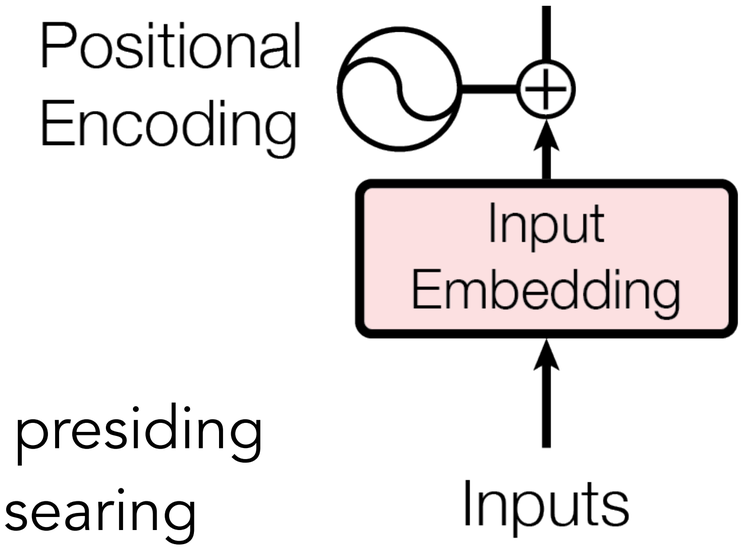
$$PE_{pos\ 2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{pos\ 2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$



- the sine function will be applied to the even numbers, and the cosine function to the odd numbers.

positional encoding



- let us consider the sentence
  - "Former president Donald Trump assailed the family judge presiding over his arraignment and District Attorney Alvin Bragg in a searing address on Tuesday after returning from his court appearance in New York earlier in the day."
- arraignment has index 11, and court 27

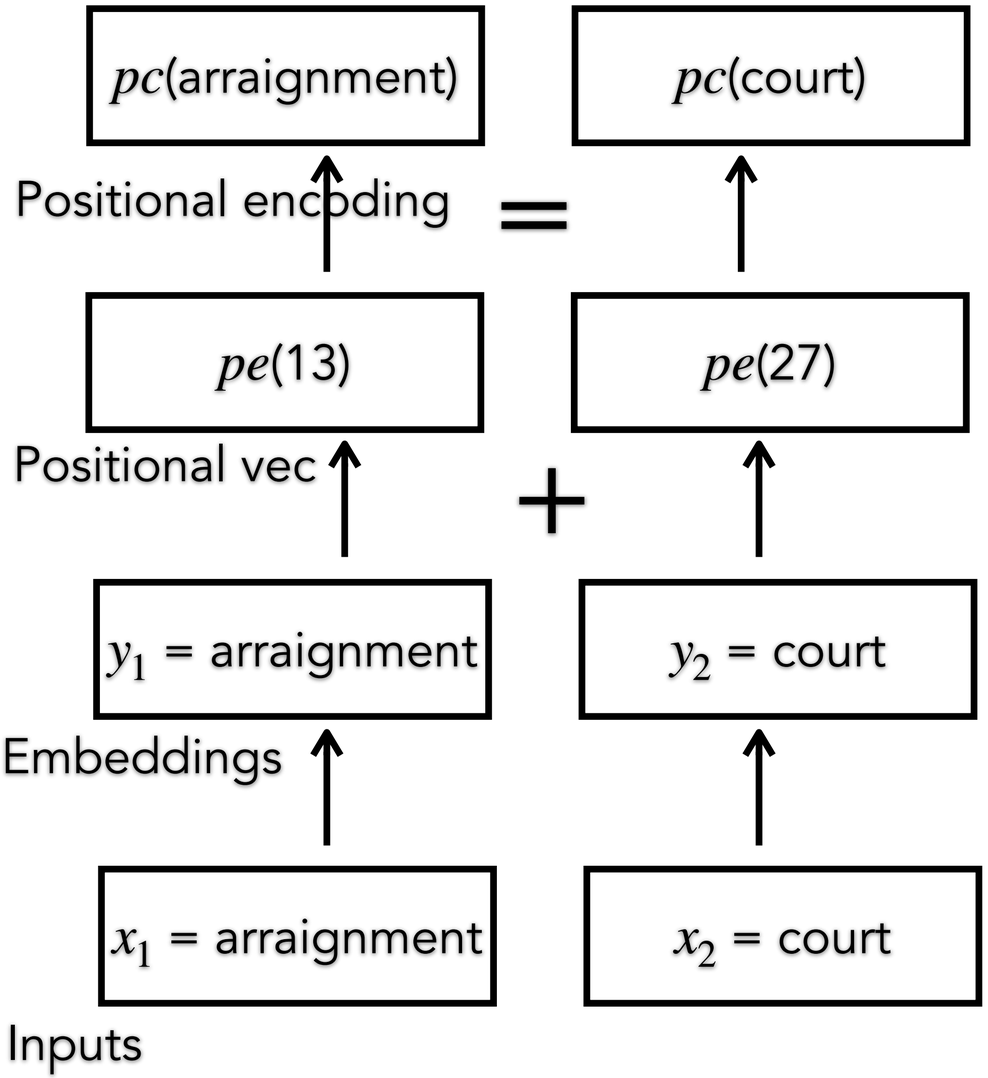
tokens	[CLS]	Former	president	...	a	##rra	##ignment	...	returning	from	his	court	...
word_ids	None	0	1	...	11	11	11	...	24	25	26	27	...
input_ids	101	6963	2084	...	170	10582	24571	...	3610	1121	1117	2175	...

positional encoding

- the PE score may be directly added to the word embedding vector:
  - e.g., we take the word embedding for 'court' (having index 27), and we add the positional vector $pe(27)$:

$$pc(\text{court}) = y_1 + pe(27)$$

  - issue: this way we may lose the information of the word embedding, which will be minimized by the positional encoding vector; values in y_2 may be amplified by adding an arbitrary value to y_2, such as $y_2 * math.sqrt(d_{model})$



sub-word tokenization

sub-word tokenization

- transformer networks operate over subword units, i.e., their tokens may be word fragments rather than complete words.
  - these sub-word units are generated automatically using the byte pair encoding (BPE) algorithm for word segmentation
  - this algorithm creates a symbol vocabulary, which keeps track of the allowed sub-word units. this dictionary is initialized with individual characters, including a special symbol </w> that indicates the end of a word.
  - it then iteratively counts the frequency of symbol pairs in a large text corpus, and replaces the most frequent pair with the concatenation of the two symbols, which is also added to the symbol dictionary

sub-word tokenization

`b a n k </w> o f </w> t h e </w> r i v e r </w>`

- should the most frequent sequence of chars be 't h', their merge would produce

`b a n k </w> o f </w> th e </w> r i v e r </w>`

- then, if the most frequent subsequence was 'th e', we would yield

`b a n k </w> o f </w> the </w> r i v e r </w>`

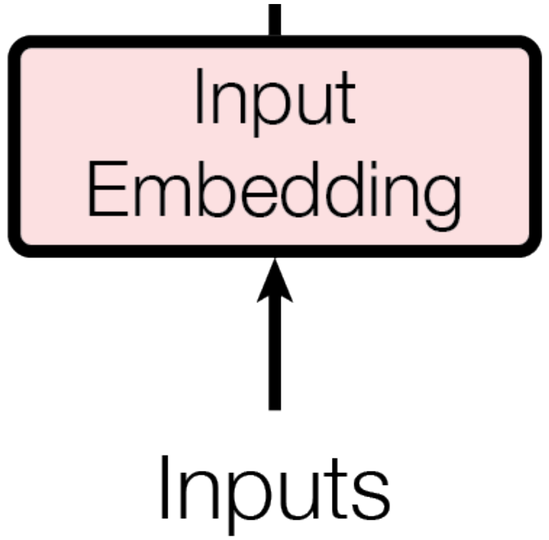
- and so forth...
- the final typical size of the vocabulary is 30-50k symbols: since this is smaller than the size of the vocabulary of any language, BPE tokenization will break infrequent words into more sub-word units.

sub-word tokenization

- sub-words make the transformer network more robust to unknown words
  - e.g., if the word 'transformers' is met for the first time after training, a traditional word embedding algorithm may not know how to handle this word, but a transformer network may still tokenize it into subwords that were seen in training such as `'transform'` and `'ers'`, for which it has trained input embeddings.
- the second advantage is saving space;
  - vocabulary size keeps growing indefinitely as the underlying text corpus grows; if each real number requires 4 bytes, each vector has 512 elements and we consider a billion words vocabulary, we would have $4 \times 512 \times 1{,}000{,}000{,}000$ (>2TB to store the input embeddings), against the cheaper $4 \times 512 \times 50{,}000$ (>100 MB to store input embeddings) for the 50k elements vocabulary

Input embedding

- this sub-layer is used to convert the input tokens to vectors of dimension $d_{model} = 512$
- the tokenizer transforms the input sentence into tokens.
  - each tokenizer has its methods, but the results are similar.



attention (is all you need!)

multi-head attention

- inside each head h_n of the attention mechanism, each word vector has three representations:
  - a query vector (Q) that has a dimension of $d_q = 64$, which is activated and trained when a word vector x_n seeks all of the key-value pairs of the other word vectors, including itself in self-attention
  - A key vector (K) that has a dimension of $d_k = 64$, which will be trained to provide an attention value
  - A value vector (V) that has a dimension of $d_v = 64$, which will be trained to provide another attention value
- attention is defined as 'scaled dot-product attention':

$$attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- vectors all have the same dimension, making it relatively simple to use a scaled dot product to obtain the attention values for each head and then concatenate the output Z of the 8 heads.

multi-head attention

- transformer networks further expand the self attention layer by repeating it multiple times
  - a typical configuration includes eight different instances of the above algorithm
  - each of these instances is called a "head"

self attention

- for each word w_i, this layer produces an output embedding, $\mathbf{z}_i$, that is a combination of all input embeddings $\mathbf{x}$ (i.e., the embeddings produced by the previous component that delivers positional information), for all the words in the input text
- three vectors of real values are employed to generate the output embeddings $\mathbf{z}_i$ by the self attention layer:
  - a query vector, $\mathbf{q}_i$
  - a key vector, $\mathbf{k}_i$
  - a value vector, $\mathbf{v}_i$
  - query and key vectors are used to generate unique attention weights for each pair of words w_i and w_j

- the intuition behind this attention mechanism can be explained with a real-world analogy: suppose you are at bakery and are interested in buying a bagel.
  - the **query** is "bagel"; and
  - the **keys** are the names of all products in the bakery, e.g., "bread" or "croissant";
  - the **value** of each product is its price.
- the goal is to pay more attention to the prices (or **values**) of the products (**keys**) that are closest to our interest (**query**).
  - in our context, the dot product $\mathbf{q}_i \cdot \mathbf{k}_j$ indicates how important word w_j is for the output embedding of word w_i.
  - the value vector $\mathbf{v}_i$ is a projection (or transformation) of the input vector $\mathbf{x}_i$ into a new feature space.
- each output embedding $\mathbf{z}_i$ will be a weighted average of these value vectors

(1) for each pair of words, w_i and w_j, compute the attention weight a_{ij} using the $\mathbf{q}_i$ and $\mathbf{k}_j$ vectors. In particular:

(a) initialize the attention weights a_{ij} with the product of the corresponding query and key vectors: $a_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$.

(b) divide the above values by the square root of the length of the key vector: $a_{ij} = \frac{a_{ij}}{\sqrt{|\mathbf{k}_1|}}$ (all $\mathbf{k}_i$ vectors have the same size, so we use $\mathbf{k}_1$ here).

(2) for each word w_i, apply softmax on all its attention weights, a_{ij}.

(3) for each word w_i, multiply the above attention weights with the corresponding value vectors, for all words w_j. then sum up all these weighted vectors to produce the output vector for w_i: $\mathbf{z}_i = \sum_j a_{ij}\mathbf{v}_j$.

1 (a) Compute all the $\mathbf{q}_1 \cdot \mathbf{k}_i$ dot products for the four words in the text:

[$\mathbf{q}_1$ bank; $\mathbf{k}_1$ bank] $a_{11} = \mathbf{q}_1 \cdot \mathbf{k}_1 = 40$

[$\mathbf{q}_1$ bank; $\mathbf{k}_2$ of] $a_{12} = \mathbf{q}_1 \cdot \mathbf{k}_2 = 16$

[$\mathbf{q}_1$ bank; $\mathbf{k}_3$ the] $a_{13} = \mathbf{q}_1 \cdot \mathbf{k}_3 = 8$

[$\mathbf{q}_1$ bank; $\mathbf{k}_4$ river] $a_{14} = \mathbf{q}_1 \cdot \mathbf{k}_4 = 32$

computing the $\mathbf{z}_i$ vector for *bank* in 'bank of the river'

---

1 (b) Divide all the above values by $\sqrt{|\mathbf{k}_1|} = 8$:

$$a_{11} = 5$$
$$a_{12} = 2$$
$$a_{13} = 1$$
$$a_{14} = 4$$

---

2 Apply softmax on the above 4 values:

$$a_{11} = 0.70$$
$$a_{12} = 0.03$$
$$a_{13} = 0.01$$
$$a_{14} = 0.26$$

softmax converts weights into a probability distribution

---

3 Compute the contextualized embedding $\mathbf{z}_1$ for *bank* as a weighted average of the value vectors:

$$\mathbf{z}_1 = 0.70\mathbf{v}_1 + 0.03\mathbf{v}_2 + 0.01\mathbf{v}_3 + 0.26\mathbf{v}_4$$

borrowed from cited Surdeanu & Valenzuela-Escárcega (2022).

parameters and settings

- each self attention block contains three matrices, $\mathbf{W^Q}$, $\mathbf{W^K}$, $\mathbf{W^V}$, such that $\mathbf{W^Q}$ and $\mathbf{W^K}$ have size $|\mathbf{x}| \times |\mathbf{k}|$, and $\mathbf{W^V}$ has size $|\mathbf{x}| \times |\mathbf{v}|$. each $\mathbf{q}_i$, $\mathbf{k}_i$ and $\mathbf{v}_i$ vector is computed as
  - $\mathbf{q}_i = \mathbf{x}_i \times \mathbf{W^Q}$,
  - $\mathbf{k}_i = \mathbf{x}_i \times \mathbf{W^K}$, and
  - $\mathbf{v}_i = \mathbf{x}_i \times \mathbf{W^V}$.
- then the parameters of the self attention layer are the three matrices
  - a typical configuration is $|\mathbf{x}| = 512$, and $|\mathbf{k}| = |\mathbf{v}| = 64$

training transformers

training

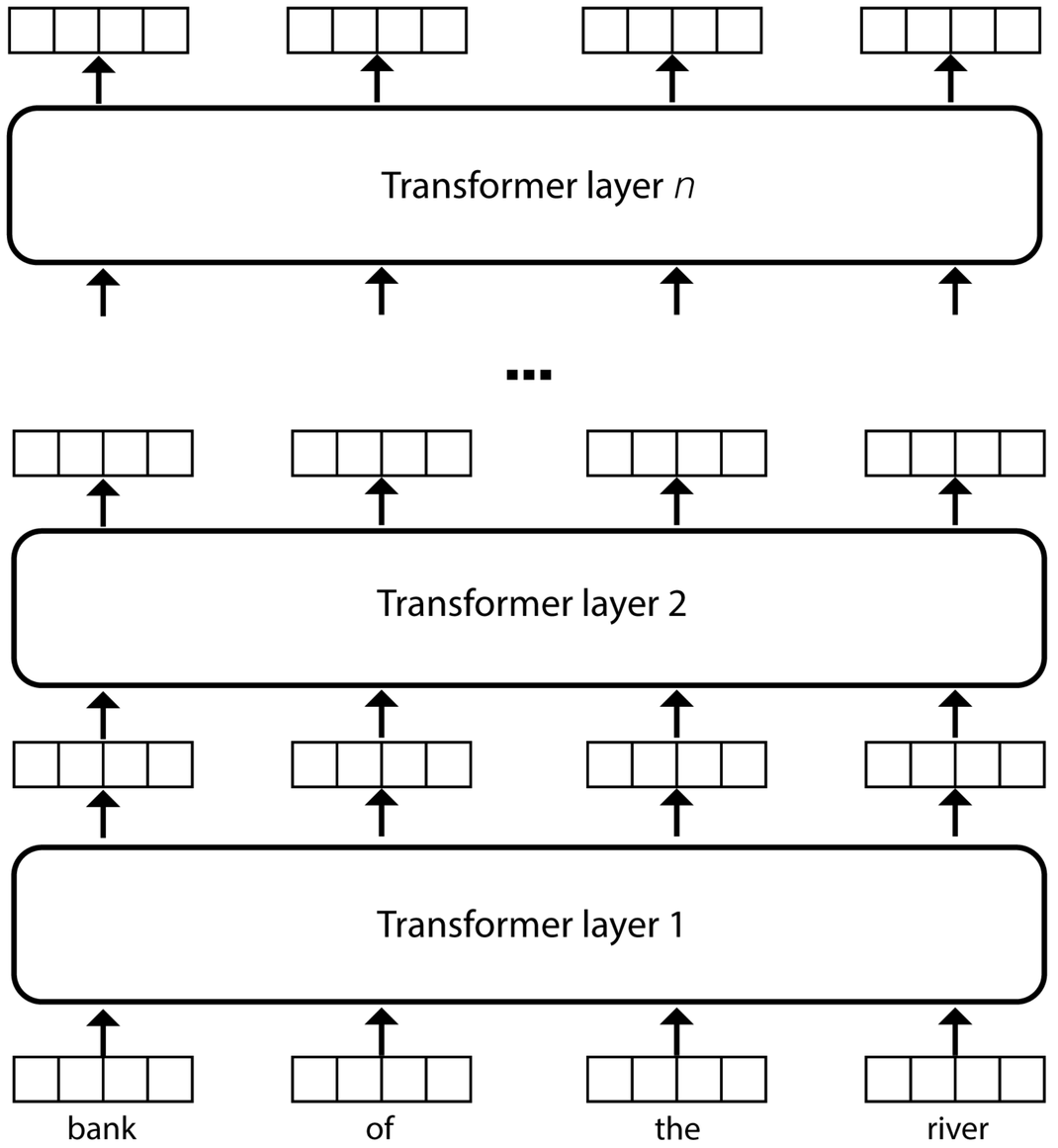
- training transformers means learning their parameters, such as the input embeddings for the subword tokens, and the various $\mathbf{W}$ matrices in each layer
  - the training process for transformer networks consists of two steps: one unsupervised procedure called pre-training, and a supervised one called fine-tuning

pre-training (MLM)

- pre-training procedures allow transformer networks to capture a variety of language patterns that are application independent
- pre-training employs a masked language model (MLM) objective
  - during pre-training tokens in some input text are masked by replacing them with a special token, e.g., [MASK], and the transformer network is asked to guess the token behind the mask.
  - typically, 15% input tokens are masked in the training texts

pre-training (MLM)

1. the classifier that predicts the masked word acts on top of the contextualized embedding produced by layer *n* for the word '*river*'.
   - the classifier itself is implemented as a softmax layer that produces a probability distribution over all the tokens in the BPE vocabulary
2. pre-training is often referred to as an unsupervised algorithm because it does not require any annotated training data from domain experts

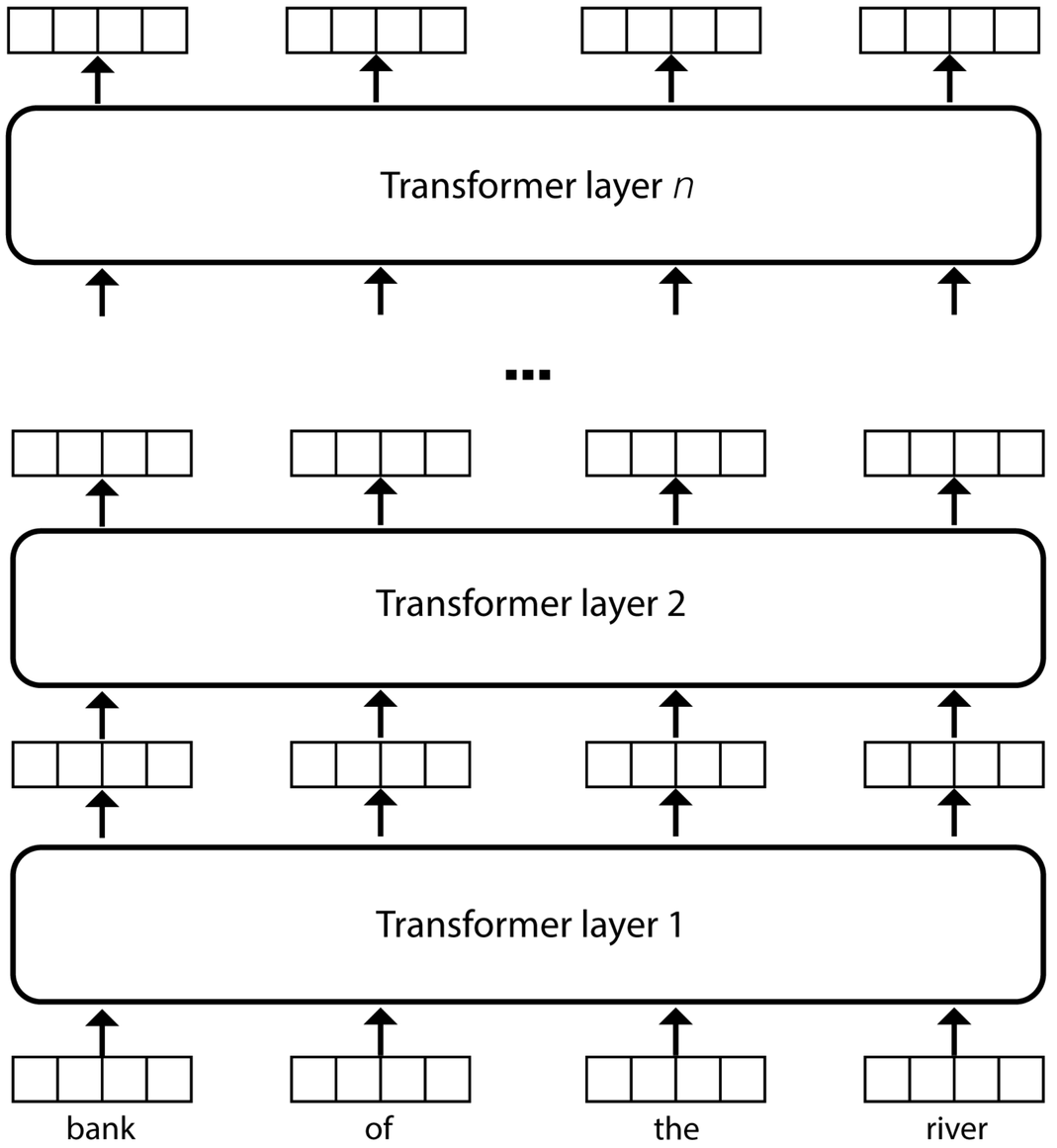


borrowed from cited Surdeanu & Valenzuela-Escárcega (2022).

pre-training (MLM)

3. masked tokens can appear anywhere in a given text; a context that can be used to guess the masked token both to the left and to the right of the mask, this pre-training procedure is called a bidirectional language model

- traditionally, language modeling (LM) tasks proceed from left to right



[CLS],I,am,the,wa,##l,##rus,.[SEP]

pre-training (NSP)

- a second pre-training procedure is next sentence prediction (NSP)
  - in this task tansformers are trained to predict which sentence follows a given input sentence
- also in this case the task is unsupervised
- the input embeddings for NSP sum up three embeddings:
  - token and position embeddings (similar to the original architecture) and
  - a new embedding that encodes which segment the current token belongs to (A or B).

[CLS],I,am,the,wa,##l,##rus,.[SEP]

pre-training (NSP)

- this architecture introduces the virtual [CLS] token, which is inserted at the beginning of the text, and whose contextualized embedding is used to train the binary NSP classifier
  - the actual classifier is a sigmoid on top of the contextualized embedding for [CLS]
  - the [CLS] token is treated like any other token in the text, i.e., its attention weights cover all the tokens in the text
  - the classifier that operates on top of the [CLS] contextualized embedding has indirect access to the entire text through its attention mechanism

[CLS],I,am,the,wa,##l,##rus,.[SEP]

pre-training (NSP)

- to train the classifier, NSP generates positive examples from actual sentences that follow a given sentence, and negative examples from sentences randomly sampled from the corpus.
  - the ratio of negative to positive examples is 1:1.

fine-tuning

- whereas pre-training procedures allow transformer networks to capture a variety of language patterns that are application independent,
- fine-tuning allows training transformer networks for specific NLP applications such as text classification, question answering, natural language inference, etc.
  - in such settings, one or more texts (separated by [SEP]) are preceded by a [CLS] token, which drives the actual classification

```
[CLS] This is a sentence[SEP] For train ##ing [SEP]
```

fine-tuning

- in fine-tuning, the training process receives a transformer network that was pre-trained using either MLM or NSP algorithm
  - the training continues in fine tuning with a loss function that is specific to the task at hand.
  - for example, for text categorization, the loss might be cross-entropy

drawbacks

- although TNs are widely adopted, they have some drawbacks:
  - the attention algorithm has quadratic runtime
  - this runtime burden is customarily mitigated through the parallelism offered by GPUs. however, when GPUs are not available, TNs are considerably slower than simpler networks (such as, e.g. recurrent neural networks)

drawbacks

- the positional embeddings in the original transformer architecture encode absolute token positions in the text.
  - e.g., in the case of syntactic parsing, what information does position 17 provide to grasp the syntactic role of a given token?
  - novel architectures encode the relative positions between any two tokens in the self-attention mechanism.
    - e.g., the position of a noun relative to a verb provides hints of its subject or object role

Exploring Lexical Semantics with DistilBERT

overview

- Main goal is to explore how transformer-based language models, specifically `distilbert-base-uncased`, can be applied to lexical semantic tasks.
  - More specifically, we are interested in analyzing word meanings in context, perform word similarity assessments, and in applying neural models for word sense disambiguation (WSD).

A: Setting Up DistilBERT for Embedding Extraction — WiC task

A: Setting Up DistilBERT for Embedding Extraction — WiC task

- Load `distilbert-base-uncased` using Hugging Face's transformers library.
- Tokenize a set of sentences and extract word embeddings from the model's hidden layers.
- Compare the embeddings of a target word in different sentences (e.g., those listed in the Appendix.txt file) to see if their vector representations capture different meanings.
- Compute cosine similarity between embeddings of "bank" in all contexts. Explore whether averaging vector for a target term form same set of sentences is beneficial to find semantic commonalities. Discuss whether the model differentiates meanings effectively.
- Use the functions developed above to solve the WiC task.

B: WiC task Using Contextual Embeddings and LLMs

B: WiC task Using Contextual Embeddings and LLMs

- Implement a simple WSD algorithm using DistilBERT:
  - By using external LLMs (e.g., Gemini, Mistral, or Llama) generate multiple example sentences for the ambiguous word (e.g., "bat" as an animal vs. a sports object).
  - Use DistilBERT to encode the new sentences and retrieve the contextualized vector for the polysemous terms: build an averaged representation for the ambiguous term.
- Use the obtained averaged representation to solve the WiC task.

C: Word Sense Disambiguation (WSD) Using Contextual Embeddings

C: WSD Using Contextual Embeddings

- Go back to the first lab assignment on SemCor, and solve the WSD task by employing the above raw algorithm (use DistilBERT to generate the contextual embeddings describing the target term).
- Compute the accuracy obtained through the above procedure.


STUDII
TAURINENSIS
1404